

Webhook Listener Service

Architecture & Implementation Snapshot

Repository: webhook-listener-service (branch: main)

Report date: 17 August 2026

Reflects commit: bbef092f — "Project structure + local infrastructure"

Scope: Full read-through of every file currently in the repository, the design recorded in each layer's README, and how the pieces are wired together.

PHASE 1 OF 11 COMPLETE — FOUNDATION & LOCAL INFRASTRUCTURE

This report describes the repository as it exists today, not the target end-state. Sections covering unbuilt layers are marked "Planned" and are based on the design already committed to each layer's README.md.

Contents

1. Executive Summary			
2. What This Service Is	External context · scope · non-goals		
3. Layered Architecture & Design Patterns	Directory layout, responsibilities, Adapter / Factory / Repository patterns		
4. File-by-File Walkthrough	4.1 Entry point & runtime · 4.2 Configuration layer · 4.3 Cross-cutting concerns	4.4 Planned business-logic layers · 4.5 Project & infrastructure files	
5. How the Pieces Talk to Each Other	5.1 Today's wiring (startup, shutdown, health/ready) · 5.2 The designed end-to-end pipeline		
6. Notable Design Decisions			
7. Current Status & Roadmap			

Executive Summary

Webhook Listener is an event-ingestion microservice for the CodePulse automated code-review platform. Its entire job is to sit at the edge of the system, receive GitHub and GitLab webhook calls, prove they are genuine, translate them into one internal event shape, and hand them off to RabbitMQ so a separate Analysis/Orchestration service can pick up the work. It deliberately never inspects or analyzes code itself — it is a gateway and a safety layer, not the product.

The repository is being built in eleven planned phases (see the roadmap in the project README). As of this snapshot, **only Phase 1 — “Project structure + local infrastructure” — is complete**. That phase delivered a working operational skeleton: an Express application with liveness/readiness endpoints, a typed environment-configuration module, a PostgreSQL connection pool, a RabbitMQ connection, centralized error handling, graceful shutdown, and a Docker Compose stack for local development. Every file that exists today supports the service *running*; none of it yet performs webhook-specific business logic.

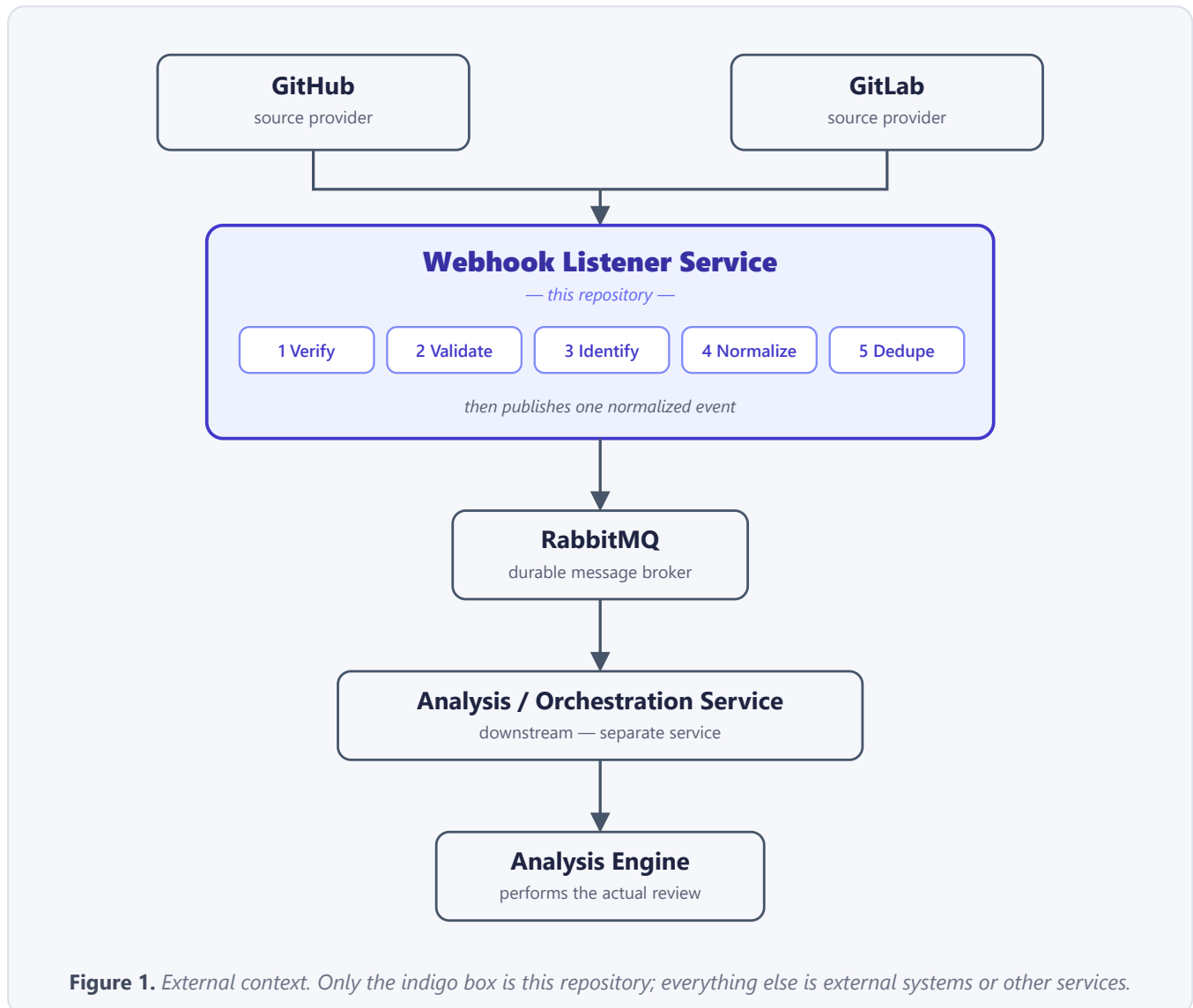
The remaining ten phases — domain event models, the provider adapter/factory abstraction, signature verification, controllers and routes, event normalization, idempotency, the RabbitMQ publisher, retry/DLQ handling, testing, and deployment configuration — are not yet implemented. Their intended shape is, however, already documented: every empty directory under `src/` contains a `README.md` that specifies what will live there and which design pattern it follows. This report treats those READMEs as a design specification and explains, file by file, both what exists today and what is planned to connect to it.

How to read this report

Section 3 explains the layering and the three design patterns (Adapter, Factory, Repository) the codebase commits to. Section 4 is the deep, file-by-file pass. Section 5 traces the actual call paths — both the ones that run today and the ones the codebase is wired to support once the remaining phases land. Every implemented file is marked **IMPLEMENTED**, every documented-but-unbuilt layer is marked **PLANNED** with its phase number, and supporting configuration is marked **CONFIG**.

What This Service Is

Webhook Listener is one hop in a longer pipeline. GitHub and GitLab are the only two systems that call it directly; RabbitMQ and the Analysis/Orchestration service are the only things it calls. Figure 1 shows that external context, including the five-step job the service performs internally before anything is published.



In scope

- Receiving HTTP webhook calls from GitHub and GitLab.
- Proving a call actually came from that provider (signature/token verification).
- Recognizing the event type and normalizing it into one internal shape regardless of source.
- Rejecting or ignoring duplicate deliveries (providers retry webhooks on timeout).
- Publishing exactly one durable message per genuine, new event to RabbitMQ.
- Reporting its own liveness and readiness so it can run safely behind a load balancer.

Explicitly out of scope

- **No code analysis.** The README states this directly: “this service never analyzes code.”
- **No synchronous coupling to the Analysis/Orchestration service.** The hand-off is a queue publish, not a call the webhook request waits on.
- **No general-purpose datastore.** Postgres is used narrowly, for idempotency bookkeeping only.

SECTION 3

Layered Architecture & Design Patterns

The codebase is organized as one directory per responsibility under `src/`. Each layer is only allowed to know about the layer(s) directly below it — controllers never parse provider payloads, adapters never touch HTTP, and nothing but `messaging/` is allowed to know RabbitMQ exists. That discipline is already visible even though most directories currently hold nothing but a README.

LAYER	DIRECTORY	RESPONSIBILITY	STATUS
Entry / runtime	<code>app.ts</code> , <code>server.ts</code>	Express app assembly, process lifecycle	IMPLEMENTED
Configuration	<code>config/</code>	Env vars, Postgres pool, RabbitMQ connection	IMPLEMENTED
Cross-cutting	<code>errors/</code> , <code>middleware/</code> , <code>types/</code>	Error vocabulary, global error handling, type augmentation	IMPLEMENTED
Controllers	<code>controllers/</code>	Thin HTTP entry points for <code>POST /webhooks/*</code>	PLANNED · PHASE 5
Factories	<code>factories/</code>	Select the right provider adapter (Factory Pattern)	PLANNED · PHASE 3
Adapters	<code>adapters/</code>	Per-provider verify + normalize (Adapter Pattern)	PLANNED · PHASES 3–4, 6
Models	<code>models/</code>	Internal domain event shapes	PLANNED · PHASE 2
Services	<code>services/</code>	Orchestrates verify → normalize → dedupe → publish	PLANNED · PHASES 4–8
Repositories	<code>repositories/</code>	Postgres access for idempotency (Repository Pattern)	PLANNED · PHASE 7
Messaging	<code>messaging/</code>	RabbitMQ topology + publisher	PLANNED · PHASES 8–9

The three named design patterns

Adapter `src/adapters/`

GitHub and GitLab send structurally different payloads, headers, and signature schemes for what is conceptually the same event (“a pull request changed”). A `GitHubAdapter` and a `GitLabAdapter` will each implement one shared interface — something shaped like `verifySignature()` plus `normalize()` — so every provider-specific quirk is contained in exactly one file per provider. Adding a third provider later means writing one new adapter, not touching the controller or service.

Factory `src/factories/`

Something has to decide *which* adapter handles a given request. Centralizing that choice in a `ProviderHandlerFactory.create(provider)` means the controller never contains an `if (provider === "github")` branch itself — it just asks the factory and gets back an object that satisfies the adapter interface.

Repository `src/repositories/`

All raw SQL is meant to live behind repository classes, never inline in services or controllers. The one repository currently planned, `ProcessedEventRepository`, answers exactly one question — “have we already published this provider delivery ID?” — keeping Postgres usage narrow and auditable rather than becoming a general-purpose store.

Why this matters for a webhook gateway specifically

Providers retry undelivered webhooks and occasionally send the same delivery twice. A layered design where normalization (adapters), orchestration (services), and persistence (repositories) are separate makes the dedup check a single well-defined step in the pipeline rather than logic scattered across the codebase.

File-by-File Walkthrough

This section covers every file in the repository. The legend below applies to every badge in this section and the table above.

- Implemented — working code in the current commit
- Planned — designed in its README, not yet coded
- Config — supporting infrastructure/tooling, not phase-gated

webhook-listener-service/	
└─ src/	
└─┬─ app.ts	Express app assembly
└─┬─ server.ts	process entry point
└─┬─ config/	
└─└─┬─ env.ts	typed process.env access
└─└─┬─ database.ts	Postgres pool
└─└─└─ rabbitmq.ts	RabbitMQ connection
└─ errors/AppError.ts	base error class
└─ middleware/errorHandler.ts	global error handler
└─ types/express.d.ts	Request.rawBody typing
└─ controllers/README.md	planned — Phase 5
└─ factories/README.md	planned — Phase 3
└─ adapters/README.md	planned — Phases 3-4, 6
└─ models/README.md	planned — Phase 2
└─ services/README.md	planned — Phases 4-8
└─ repositories/README.md	planned — Phase 7
└─┬─ messaging/README.md	planned — Phases 8-9
└─ database/migrations/.gitkeep	empty — first migration in Phase 7
└─ dist/	compiled src/, gitignored
└─ docker-compose.yml	Local Postgres + RabbitMQ
└─ database.json	node-pg-migrate config
└─ package.json / tsconfig.json	build & scripts
└─ .env.example	local env template
└─ README.md	project overview + roadmap

4.1 Entry Point & Runtime

The process entry point (`package.json`'s `main` and the target of `npm run dev / start`). Its `main()` function connects to Postgres and RabbitMQ *before* opening the HTTP port, then registers signal handlers for a graceful shutdown.

src/server.ts – shutdown ordering

```
const shutdown = (signal: string): void => {
  server.close(() => {
    void (async () => {
      await closeRabbitMQ();
      await pool.end();
      process.exit(0);
    })();
  });
};
```

- Connects DB, then RabbitMQ, then starts listening — in that order, so the service never accepts traffic it cannot back with durable infrastructure.
- On SIGTERM/SIGINT: stop accepting new HTTP connections → let in-flight ones finish → close the RabbitMQ channel/connection → drain the Postgres pool → exit. This ordering exists specifically for Kubernetes, which sends SIGTERM before killing a pod.

TALKS TO

[app.ts](#)[config/env.ts](#)[config/database.ts](#)[config/rabbitmq.ts](#)

Builds and configures the Express application object but does not call `listen()` itself — that separation keeps the app importable/testable independently of the running process. Three things live here today: the global JSON body-parser, the `/health` and `/ready` endpoints, and the error handler.

src/app.ts – capturing the raw body for future signature checks

```
app.use(express.json({
  limit: "2mb",
  verify: (req, _res, buf) => {
    (req as express.Request).rawBody = buf;
  },
}));
```

- The body-size limit (2 MB) exists because this endpoint is reachable from the public internet — an unbounded body is a denial-of-service vector.
- The `verify` hook stashes the exact request bytes on `req.rawBody` before Express parses them into an object. Future HMAC signature verification (Phase 4) must hash the *exact* bytes the provider signed — a parsed-then-re-serialized JSON object is not guaranteed to be byte-identical.
- `/health` does no dependency checks at all (liveness only). `/ready` queries both Postgres and RabbitMQ and only returns `200` if both are reachable (readiness). A comment in the file marks exactly where the future webhook routes will mount, between these endpoints and the error handler.

TALKS TO

[middleware/errorHandler.ts](#)

[config/rabbitmq.ts](#)

[config/database.ts](#)

4.2 Configuration Layer

The single place `process.env` is read. It imports `dotenv/config` for its side effect (loading `.env`) and exports one frozen-shape `env` object covering `NODE_ENV`, `PORT` (default `5002`), the five `DB_*` variables, and `RABBITMQ_URL`. Every other module imports `env` rather than touching `process.env` directly.

- The `DB_*` fields use TypeScript's non-null assertion (`process.env.DB_HOST!`). This satisfies the type checker (the field is typed `string`, not `string | undefined`) but performs no runtime validation — if `.env` is missing a value, `env.DB_HOST` is silently undefined at runtime until something downstream (the Postgres pool) fails on it.
- Explicitly written to mirror the sibling `integration-service` codebase's `config/env.ts`, so an engineer moving between the two services reads familiar structure.

CONSUMED BY

[config/database.ts](#)

[config/rabbitmq.ts](#)

[server.ts](#)

src/config/database.ts

IMPLEMENTED

Creates the one shared `pg.Pool` (max 10 connections, 30s idle timeout, 5s connect timeout) that every future data access will reuse, and exposes `connectDatabase()` as a startup sanity check.

src/config/database.ts – fail-fast on startup

```
export async function connectDatabase(): Promise<void> {
  try {
    const result = await pool.query("SELECT NOW();");
    console.log("Webhook Listener connected to PostgreSQL.", result.rows[0]);
  } catch (error) {
    console.error("Failed to connect to PostgreSQL:", error);
    process.exit(1);
  }
}
```

- **Fail-fast:** if Postgres is unreachable at boot, the process exits immediately rather than serving webhook traffic it cannot durably record.
- The comment in this file is explicit that Postgres is used here only where persistence is genuinely required (idempotency tracking) — not as a general datastore.

TALKS TO [config/env.ts](#) USED BY [server.ts](#) [app.ts \(/ready\)](#) [repositories/ \(planned\)](#)

src/config/rabbitmq.ts

IMPLEMENTED

Owns RabbitMQ *connection lifecycle only* — a module-level connection and one shared channel, both undefined until `connectRabbitMQ()` runs. Exchange/queue topology and publishing are explicitly left to the future `messaging/` layer; this file's only job is answering "are we connected."

- `connectRabbitMQ()` — same fail-fast pattern as the database module: `process.exit(1)` if the broker can't be reached at boot.
- `getRabbitMQChannel()` — throws if called before `connectRabbitMQ()` completes, guarding against use-before-init. This is the accessor the future publisher will call.
- `isRabbitMQConnected()` — the boolean `/ready` reads.
- `closeRabbitMQ()` — closes channel then connection; called during graceful shutdown.

TALKS TO [config/env.ts](#) USED BY [server.ts](#) [app.ts \(/ready\)](#) [messaging/ \(planned\)](#)

4.3 Cross-Cutting Concerns

src/errors/AppError.ts

IMPLEMENTED

A 12-line base class: `AppError` extends `Error`, adding a `statusCode` (default 500) and explicitly restoring the prototype chain via `Object.setPrototypeOf` so `instanceof AppError` checks work correctly after transpilation. This is the vocabulary every future layer will use to signal an *expected* failure with a specific HTTP status — e.g. a future `InvalidSignatureError` or `UnsupportedProviderError` would subclass this.

CONSUMED BY `middleware/errorHandler.ts`

src/middleware/errorHandler.ts

IMPLEMENTED

The global Express error-handling middleware (recognizable to Express by its four-argument signature), mounted last in `app.ts` as required. If the thrown error is an `AppError`, it responds with that error's own status code and message. For anything else, it logs the full error server-side and responds with a generic 500 — deliberately withholding internal details, since this is an internet-facing endpoint reachable by external webhook senders.

A comment already earmarks future work: once structured logging and correlation/delivery IDs exist (attached upstream), they will be included here so a failure can be traced back to one specific provider delivery.

TALKS TO `errors/AppError.ts` REGISTERED BY `app.ts`

src/types/express.d.ts

IMPLEMENTED

A pure ambient type declaration — no runtime code, and correspondingly no file appears in `dist/` for it. It declaration-merges an optional `rawBody?: Buffer` field onto Express's Request interface so that `req.rawBody` (set in `app.ts`) type-checks everywhere it is written or read.

SET BY `app.ts` WILL BE READ BY `adapters/ (planned, Phase 4)`

4.4 Planned Business-Logic Layers

Nothing below yet contains code — each directory holds only a `README.md`. They are documented here in the order data will actually flow through them, expanding on what each `README` commits to.

src/controllers/

PLANNED · PHASE 5

HTTP entry points — `POST /webhooks/github` and `POST /webhooks/gitlab` — kept deliberately thin: parse headers/params, ask the factory for the right adapter, delegate to the service layer, and return quickly. The `README` is explicit that this layer must **not** synchronously wait on the Analysis/Orchestration service and must contain no GitHub/GitLab API calls or analysis logic.

WILL CALL `factories/` `services/` MOUNTED FROM `app.ts`

src/factories/

PLANNED · PHASE 3

A single planned entry point, `ProviderHandlerFactory.create(provider)`, that inspects the route or a provider-identifying header and returns the matching adapter instance. Its entire purpose is keeping provider branching out of the controller.

WILL RETURN `adapters/` **CALLED BY** `controllers/`

src/adapters/

PLANNED · PHASES 3–4, 6

One handler per provider (`GitHubAdapter`, `GitLabAdapter`), both implementing a shared `ProviderWebhookHandler` interface. Each is responsible for verifying the request actually came from that provider — GitHub's HMAC-SHA256 over X-Hub-Signature-256 using the raw body captured in `app.ts`, GitLab's secret-token header — and for normalizing that provider's payload shape into the internal `models/` representation.

READS `req.rawBody` **PRODUCES** `models/` **SELECTED BY** `factories/`

src/models/

PLANNED · PHASE 2

The internal domain shapes every provider payload gets normalized into — a generic `WebhookEvent` envelope (provider, event type, delivery ID, repository identity, timestamp, raw-payload reference) and a `PullRequestEvent`, the common representation both a GitHub `pull_request` event and a GitLab Merge Request event collapse into. Downstream consumers — RabbitMQ, and ultimately the Analysis/Orchestration service — only ever see these shapes, never a raw provider payload. New event types (push, issue, deployment, ...) are meant to be added as new model shapes without touching the controller.

PRODUCED BY `adapters/` **CONSUMED BY** `services/` `messaging/`

src/services/

PLANNED · PHASES 4–8

The orchestration layer: verify → normalize → deduplicate → publish. It coordinates adapters, repositories, and messaging, and is where the pipeline steps shown in Figure 1 actually get sequenced. It contains no HTTP concerns (the controller's job) and no provider-specific parsing (the adapters' job) — introduced incrementally as each phase lands rather than all at once.

CALLS `adapters/` `repositories/` `messaging/` **CALLED BY** `controllers/`

src/repositories/

PLANNED · PHASE 7

The only place raw SQL will live, matching integration-service's convention of using pg directly with no ORM. The one planned repository, ProcessedEventRepository, records a provider delivery ID once its event has been published, and lets the service layer check "have we already handled this one" before publishing again.

USES `config/database.ts (pool)` CALLED BY `services/`

src/messaging/

PLANNED · PHASES 8–9

RabbitMQ publisher and topology — durable exchange/queue declarations, persistent messages, routing keys, and (Phase 9) the retry / dead-letter-queue configuration protecting against lost events if the downstream Analysis/Orchestration service is temporarily unavailable. Per its README, this is the *only* layer allowed to know RabbitMQ exists; everything upstream just calls a plain `publish(event)`-shaped function.

USES `config/rabbitmq.ts (getRabbitMQChannel)` CALLED BY `services/`

4.5 Project & Infrastructure Files

package.json

CONFIG

Declares the project as an ES module (`"type": "module"`), which is why every relative import in the source uses an explicit `.js` extension even though the files are `.ts` (required under the NodeNext module resolution set in `tsconfig.json`). `npm run dev` runs `tsx watch src/server.ts` (no separate build step locally); `npm run build` runs `tsc` to `dist/`; `migrate:up/down` invoke `node-pg-migrate` directly with `dotenv/config` preloaded. Runtime dependencies are minimal and match what's actually used: `express`, `amqplib`, `pg`, `node-pg-migrate`, `dotenv`.

tsconfig.json

CONFIG

Targets ES2022 with `module/moduleResolution`: NodeNext (the ESM-with-explicit-extensions setup `package.json` requires), strict type-checking, `rootDir`: `src` → `outDir`: `dist`, and `resolveJsonModule` for the JSON config files. There is no `sourceMap` option set, which is why `dist/` contains plain `.js` with no accompanying `.map` files.

docker-compose.yml

[CONFIG](#)

Two services for local development: `postgres:16-alpine` mapped to host port **5433** (not the default 5432) specifically so this stack can run alongside a sibling `integration-service`'s native Postgres install locally; and `rabbitmq:3-management-alpine` exposing **5672** (AMQP, used by the app) and **15672** (management UI). Both declare healthchecks and named volumes so data survives container restarts.

.env.example

[CONFIG](#)

A template that already matches `docker-compose.yml`'s ports and credentials exactly, so `cp .env.example .env` works with zero edits against the local stack. Covers `NODE_ENV`, `PORT` (5002), the five `DB_*` values, and `RABBITMQ_URL`.

database.json

[+ database/migrations/](#)[CONFIG](#)

`node-pg-migrate`'s configuration: SQL-language migrations, `database/migrations` as the directory, and a connection string pointed at the Compose Postgres on port 5433.
`database/migrations/.gitkeep` is the only file in that directory today — an empty placeholder keeping the folder tracked in git until Phase 7 adds the first real migration (the idempotency-tracking table).

dist/

[CONFIG \(GENERATED\)](#)

Compiled output of `src/`, produced by `npm run build` and gitignored. It currently mirrors the seven implemented `.ts` files one-for-one (`app.js`, `server.js`, `config/*.js`, `errors/AppError.js`, `middleware/errorHandler.js`) — there is no compiled output for `types/express.d.ts` since ambient type declarations produce no JavaScript. Not hand-maintained; it should never be edited directly.

How the Pieces Talk to Each Other

5.1 Today's Wiring

Only four call paths exist right now: process startup, the two HTTP endpoints, and process shutdown. All four are already fully wired end-to-end.

Startup `npm run dev / npm start` → `server.ts: main()`

Importing `config/env.ts` (transitively, via `app.ts` and `config/database.ts`) loads `.env` as a side effect. `main()` then runs `connectDatabase()`, then `connectRabbitMQ()` — both fail-fast — and only then calls `app.listen(env.PORT)`. Finally it registers the `SIGTERM/SIGINT` handlers.

GET /health caller → `app.ts` handler → static JSON

No other module is touched. This is intentionally the cheapest possible check — “is the process alive” — so a temporarily slow database or broker never causes Kubernetes to kill and restart an otherwise-healthy pod.

GET /ready caller → `app.ts` handler → `config/database.ts` + `config/rabbitmq.ts`

The handler runs `pool.query("SELECT 1;")` and calls `isRabbitMQConnected()`, then returns 200 only if both are true, else 503. This is what a load balancer or Kubernetes readiness probe polls before routing real webhook traffic to a given instance.

Shutdown `SIGTERM/SIGINT` → `server.ts: shutdown()`

`server.close()` (stop accepting new connections, finish in-flight ones) → `closeRabbitMQ()` → `pool.end()` → `process.exit(0)`. This ordering is what prevents an in-flight webhook or an unpublished message from being dropped when a pod is recycled.

5.2 The Designed End-to-End Pipeline (Planned)

No route for an actual webhook exists yet — `app.ts` has a comment marking exactly where it will be mounted. Figure 2 shows how the planned layers are meant to call one another once Phases 2–9 land, based on each layer's README.

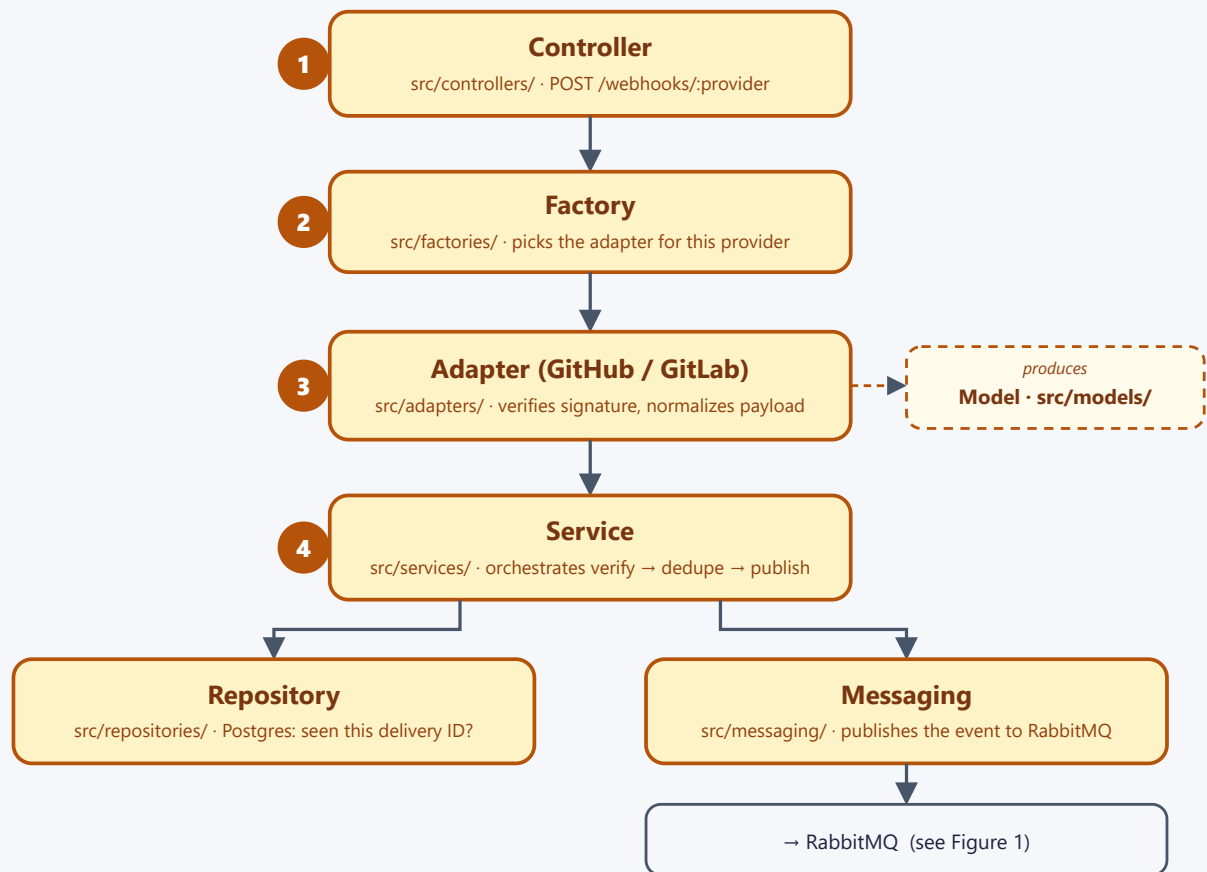


Figure 2. Planned control flow for one incoming webhook, based on the responsibilities each layer's README already commits to. All boxes are currently unbuilt (Phases 2–9). The dashed box is a data shape, not a processing step.

Written out as a sequence, one webhook delivery is meant to move through:

1. GitHub/GitLab sends POST /webhooks/github (or /gitlab).
2. app.ts's `express.json()` middleware parses the body and captures the exact bytes on `req.rawBody` — already built today.
3. The **controller** reads headers/params and asks the **factory** for a provider adapter.
4. The **factory** returns a `GitHubAdapter` or `GitLabAdapter` instance.
5. The **service** asks that **adapter** to verify the signature against `req.rawBody`, then to normalize the payload into a `WebhookEvent` / `PullRequestEvent` (**model**).
6. The **service** asks the **repository** whether this provider delivery ID (GitHub sends one in `X-GitHub-Delivery`) has already been processed. If yes, it short-circuits with a fast, idempotent response — no duplicate publish.
7. If new, the **service** calls the **messaging** publisher, which obtains the shared channel via `getRabbitMQChannel()` (already implemented in `config/rabbitmq.ts`) and publishes a durable, persistent message.
8. The **service** tells the **repository** to record the delivery ID as processed.
9. The **controller** returns a fast 2xx to the provider — it must not block on the downstream Analysis service.
10. RabbitMQ holds the message until the Analysis/Orchestration service consumes it independently.

What's genuinely already in place for this flow

Three pieces of groundwork already built will be reused as-is once the pipeline lands: the raw-body capture in `app.ts` (step 2), the `getRabbitMQChannel()` accessor in `config/rabbitmq.ts` (step 7), and the `AppError/errorHandler` pair, which will catch and format failures thrown anywhere in this chain — an invalid signature, an unsupported provider, a publish failure — without controllers or services needing their own try/catch-and-format boilerplate.

Notable Design Decisions

Several choices in the current code are easy to miss on a quick read but shape everything built on top of them.

- **Fail-fast startup, not lazy connection**

`connectDatabase()` and `connectRabbitMQ()` both call `process.exit(1)` on failure and both run before `app.listen()`. The service would rather not start at all than accept webhook traffic it cannot durably hand off downstream.

- **Raw body capture precedes signature verification by three phases**

The `express.json({ verify })` hook was built in Phase 1, well before HMAC verification (Phase 4) needs it. Capturing exact bytes has to happen at parse time — it cannot be reconstructed later from the parsed object — so this was correctly sequenced early even though nothing consumes `req.rawBody` yet.

- **/health vs /ready are deliberately different questions**

Liveness (“is the process up”) and readiness (“can this instance do its job right now”) are split so a temporarily degraded database or broker doesn’t cause an orchestrator to kill and restart an otherwise-fine process for no benefit.

- **Shutdown order: HTTP server → broker → pool**

Closing in this specific order avoids dropping an in-flight webhook request or an unpublished RabbitMQ message during the SIGTERM window Kubernetes gives a pod before killing it.

- **Non-default Postgres port (5433) by design, not accident**

The compose file maps Postgres to host port 5433 specifically so this service’s local stack can run *alongside* a sibling `integration-service`’s native Postgres on the default 5432 — a concrete signal this repository is one of several services in a larger local multi-service setup.

- **ESM + NodeNext throughout**

`package.json`’s `"type": "module"` plus `tsconfig.json`’s NodeNext resolution is why every relative import (e.g. `./app.js` inside `server.ts`) uses an explicit `.js` extension even though the source file is `.ts` — a common point of confusion for anyone used to CommonJS or bundler-based resolution, worth knowing before touching imports in this repo.

- **One centralized env object**

Every environment variable is read from `process.env` exactly once, in `config/env.ts`. The rest of the codebase imports the resulting env object rather than reading `process.env` ad hoc — one place to audit, one place to add validation later.

- **A non-leaky error vocabulary from day one**

`AppError` plus the global handler mean every future “expected” failure (bad signature, unknown provider, malformed payload) can carry its own status code, while every *unexpected* exception is automatically reduced to a generic `500` before it reaches an external caller — important because this endpoint’s callers are outside the organization.

SECTION 7

Current Status & Roadmap

The project README defines an eleven-phase build order. One phase is complete.



#	PHASE	STATUS	PRIMARILY TOUCHES
1	Project structure + local infrastructure	DONE	app.ts , server.ts , config/ , errors/ , middleware/ , Docker Compose
2	Domain event models	PLANNED	models/
3	Provider abstraction (adapter interface + factory)	PLANNED	adapters/ , factories/
4	Signature verification (GitHub HMAC-SHA256, GitLab secret token)	PLANNED	adapters/
5	Webhook controllers + routes	PLANNED	controllers/ , app.ts (route mount)
6	Event normalization (GitHub PR / GitLab MR → PullRequestEvent)	PLANNED	adapters/ , models/
7	Idempotency / duplicate-event protection	PLANNED	repositories/ , database/migrations/
8	RabbitMQ publisher (durable exchange/queue, persistent messages)	PLANNED	messaging/
9	Retry / dead-letter handling	PLANNED	messaging/
10	Testing (unit, integration, API, e2e)	PLANNED	all layers
11	Docker/AWS/Kubernetes deployment configuration	PLANNED	project root, infra

Bottom line. The repository is operationally prepared but not yet functionally complete as a webhook-processing pipeline. What exists is a correctly fail-fast, gracefully-shutting-down Express service with real Postgres and RabbitMQ connections, real health/readiness semantics, and a non-leaky error-handling story — plus a fully specified design (via the per-directory READMEs) for the provider adapters, factory selection, controllers, normalized event models, deduplication repository, and RabbitMQ publishing topology that

make up the actual business pipeline. Those seven pieces are the next phases, and this report's Section 4.4 and Figure 2 reflect exactly what they are committed to doing once built.